1. Набор полей в таблице clients в целом выглядит вполне адекватным для хранения информации о зарегистрировавшихся покупателях. Однако, я бы внесла некоторые изменения и дополнения:

   Добавила бы поле email varchar(255) для хранения электронной почты клиента, так как это важный контактный адрес.

   Добавила бы поле phone varchar(20) для хранения телефонного номера клиента.

   Изменила бы поле city_id на city_name varchar(255), чтобы хранить не только идентификатор города, но и его название. Это упростит работу с данными и делает их более понятными для аналитики.

   Добавила бы поле gender varchar(10) для хранения пола клиента.

   Добавила бы поле last_login date для отслеживания последнего входа клиента на сайт.

   Таким образом, после внесения указанных изменений таблица clients будет более информативной и удобной для работы с данными о клиентах интернет-магазина.

2. **1 вариант**

   Для того чтобы обеспечить историческое хранение стоимости товара в интернет-магазине, можно использовать подход с созданием дополнительной таблицы для отслеживания изменений цен. Это позволит сохранить историю цен на товары и проводить анализ за различные временные промежутки.

   Вот возможная схема для такой доработки:

   1. Создать новую таблицу item_prices, которая будет хранить стоимость товара в определённые временные промежутки.

```
CREATE TABLE item_prices (
   price_id NUMBER PRIMARY KEY, -- уникальный id записи цены
   item_id NUMBER, -- внешний ключ, ссылающийся на таблицу items
   item_cost NUMBER, -- стоимость товара
   start_date DATE, -- дата начала действия этой цены
   end_date DATE, -- дата окончания действия этой цены
   CONSTRAINT fk_item FOREIGN KEY (item_id) REFERENCES items(item_id)
);
```

   Пояснение полей:
   - price_id: уникальный идентификатор на каждую запись цен.
   - item_id: ссылается на `item_id` из таблицы `items` для связи товара с его ценами.
   - item_cost: собственно, стоимость товара.
   - start_date: дата, с которой данная цена начала действовать.
   - end_date: дата, до которой данная цена была актуальна. Если цена действовала до настоящего времени, данное поле можно оставить NULL или указать некую финальную дату (например, '9999-12-31').

   С помощью такой схемы можно легко получить информацию о ценах на товары за любой период и поддерживать актуальность данных в базе.

2 вариант

Добавить дополнительное поле в таблицу items, которое будет отражать дату изменения цены. Например:

items (item_id number, --уникальный id товара

item_name varchar(255), --наименование товара

item_cost number, --стоимость товара

price_date date); --дата изменения цены

Таким образом, каждый раз, когда цена товара меняется, новая запись будет добавляться в таблицу с указанием даты изменения. Это позволит сохранить историю изменения цен и использовать эту информацию для построения отчетов за разные промежутки времени. Кроме того, можно создать соответствующие индексы для повышения производительности запросов к этой таблице.

3. SELECT D.DEPARTMENT_ID, D.DEPARTMENT_NAME, SUM(E.SALARY) AS TOTAL_SALARY
FROM EMPLOYEES E
JOIN DEPARTMENTS D ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
WHERE E.MANAGER_ID IS NULL
AND D.LOCATION_ID = (SELECT LOCATION_ID FROM LOCATIONS WHERE CITY = 'SEOUL')
GROUP BY D.DEPARTMENT_ID, D.DEPARTMENT_NAME
HAVING SUM(E.SALARY) > 100000;

1. Сравнение с NULL: Чтобы проверить, является ли значение NULL, нужно использовать оператор IS NULL, а не =.

2. Условия GROUP BY и HAVING: После агрегатной функции SUM должно использоваться условие HAVING, если вы хотите фильтровать результаты группировки.

3. Фильтрация по отделам: Мы должны сначала найти все сотрудники, не имеющие менеджера, и затем посчитать их зарплату по отделам.

Объяснение исправленного запроса:

- Используем JOIN для соединения таблиц EMPLOYEES и DEPARTMENTS по DEPARTMENT_ID.

- Фильтруем сотрудников, где MANAGER_ID является NULL.

- Используем подзапрос для нахождения LOCATION_ID для города Сеул.

- Группируем результаты по DEPARTMENT_ID и DEPARTMENT_NAME.

- Используем HAVING для фильтрации групп, у которых сумма зарплаты превышает 100000.

4. Карандаш уронили у стены, я не смогла бы его перепрыгнуть потому что врезалась бы в стену.